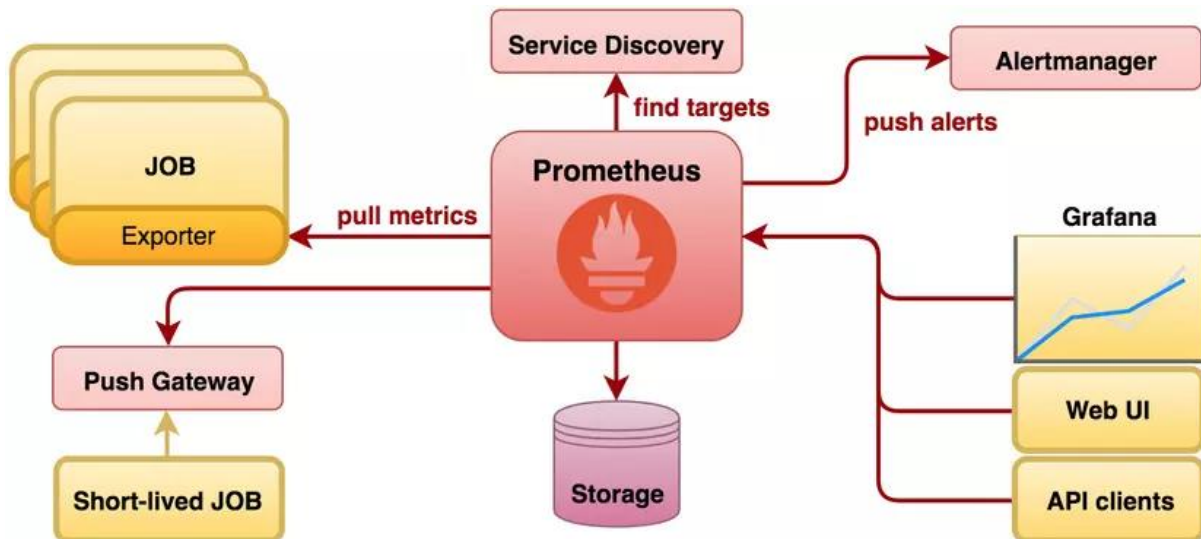


Giám sát hệ thống Docker Swarm với Prometheus, Grafana và InfluxDB



Prometheus:

node-exporter & cAdvisor export dữ liệu → Prometheus pull về định kỳ → Lưu trữ vào time-series DB → Grafana dùng để hiển thị dashboard.

Prometheus.yml:

global:

scrape_interval: 15s

scrape_configs:

- job_name: 'prometheus'

static_configs:

- targets: ['localhost:9090']

- job_name: 'node-exporter'

static_configs:

- targets:

- '192.168.56.104:9100'

- '192.168.56.105:9100'

- job_name: 'cadvisor'

static_configs:

- targets:

- '192.168.56.104:8080'

- '192.168.56.105:8080'

remote_write:

- url: "http://192.168.56.101:8086/api/v1/prom/write?db=prometheus"

basic_auth:

username: "admin"

password: "adminpassword"

Tạo config prometheus

docker config create prometheus_config ./prometheus-grafana/prometheus.yml'

Tạo prometheus:

docker service create --name prometheus \

--network prometheusnetwork \

--config source=prometheus_config,target=/etc/prometheus/prometheus.yml \

-p 9090:9090 \

prom/prometheus

Có thể kiểm tra các api của prometheus ở đây:

<https://prometheus.io/docs/prometheus/latest/querying/api/>

Tạo Grafana: Trực quan hóa dữ liệu

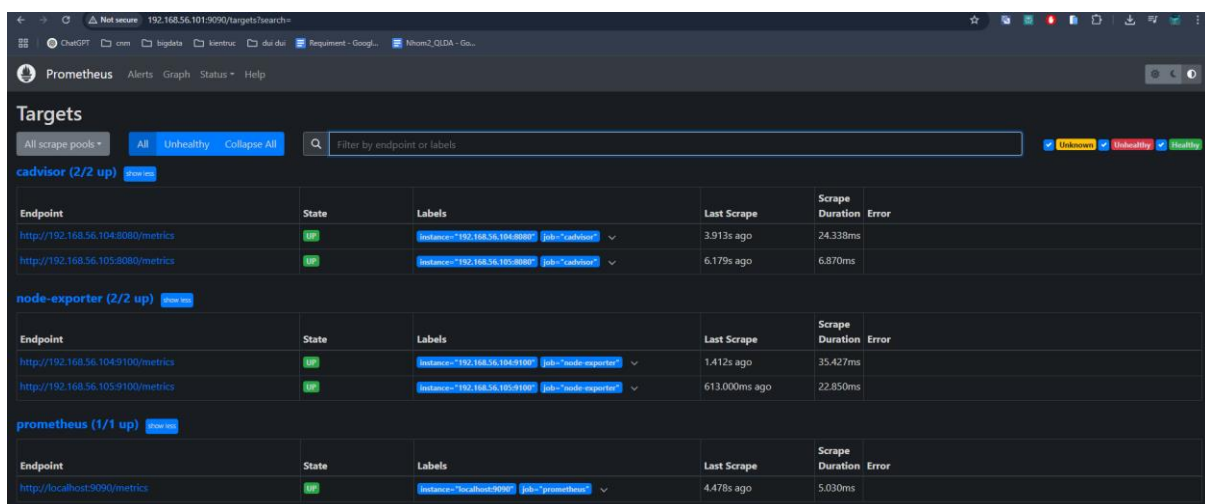
```
docker service create --name grafana --publish 3000:3000 --mount
type=volume,source=grafana_data,target=/var/lib/grafana --network coinswarmnet
grafana/grafana:latest
```

Tạo node-exporter và cadvisor để export các metrics:

```
docker service create --name node-exporter --mode global --network
coinswarmnet --mount type=bind,source=/proc,target=/host/proc,readonly --mount
type=bind,source=/sys,target=/host/sys,readonly --mount
type=bind,source=/,target=/rootfs,readonly --env NODE_ID=$(hostname) --env
HOST_PROC=/host/proc --env HOST_SYS=/host/sys --env
HOST_ROOTFS=/rootfs --publish mode=host,target=9100,published=9100
prom/node-exporter:latest --path.procfs=/host/proc --path.sysfs=/host/sys --
path.rootfs=/rootfs
```

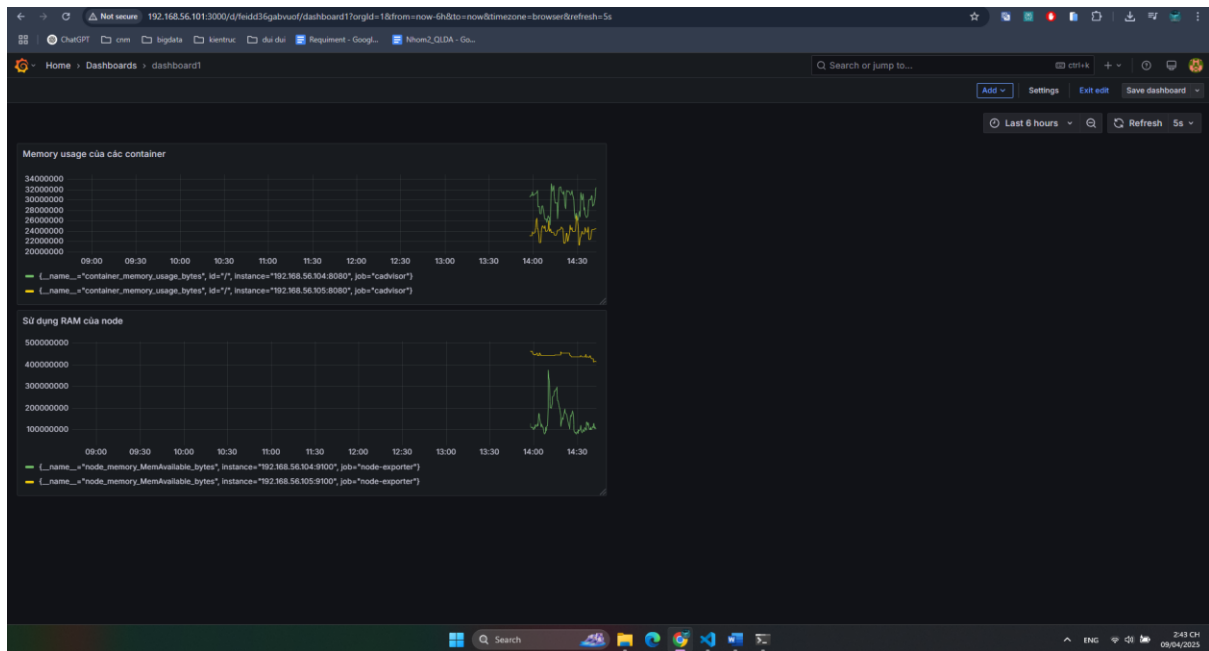
```
docker service create --name cadvisor --mode global --network coinswarmnet --
mount type=bind,source=/,target=/rootfs:ro --mount
type=bind,source=/var/run,target=/var/run:rw --mount
type=bind,source=/sys,target=/sys:ro --mount
type=bind,source=/var/lib/docker/,target=/var/lib/docker:ro --publish
mode=host,target=8080,published=8080 gcr.io/cadvisor/cadvisor:latest
```

Lúc này truy cập vào prometheus vào status -> target sẽ thấy:



Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
cadvisor (2/2 up)					
http://192.168.56.104:8080/metrics	up	instance="192.168.56.104:8080" job="cadvisor"	3.913s ago	24.338ms	
http://192.168.56.105:8080/metrics	up	instance="192.168.56.105:8080" job="cadvisor"	6.179s ago	6.870ms	
node-exporter (2/2 up)					
http://192.168.56.104:9100/metrics	up	instance="192.168.56.104:9100" job="node-exporter"	1.412s ago	35.427ms	
http://192.168.56.105:9100/metrics	up	instance="192.168.56.105:9100" job="node-exporter"	613.000ms ago	22.850ms	
prometheus (1/1 up)					
http://localhost:9090/metrics	up	instance="localhost:9090" job="prometheus"	4.478s ago	5.030ms	

Qua Grafana để trực quan hóa dữ liệu: tìm hiểu metric đó để đo gì và thực hiện trực quan hóa



Dữ liệu sẽ được lưu ở container prometheus:

Có thể thực hiện các lệnh sau để xem:

`Sudo docker exec -it 4a8ad268825c sh`

`Ls`

Nội dung thư mục /prometheus:

- chunks_head: chứa dữ liệu time-series trong bộ nhớ (in-memory).
- wal/: Write-Ahead Log – Prometheus ghi dữ liệu tạm vào đây trước khi lưu chính thức.
- queries.active: các truy vấn đang được thực hiện.
- lock: để tránh nhiều tiến trình ghi đồng thời.

Khởi tạo influxdb:

`docker service create \`

`--name influxdb \`

`--publish 8086:8086 \`

`--env INFLUXDB_DB=prometheus \`

`--env INFLUXDB_ADMIN_USER=admin \`

`--env INFLUXDB_ADMIN_PASSWORD=adminpassword \`

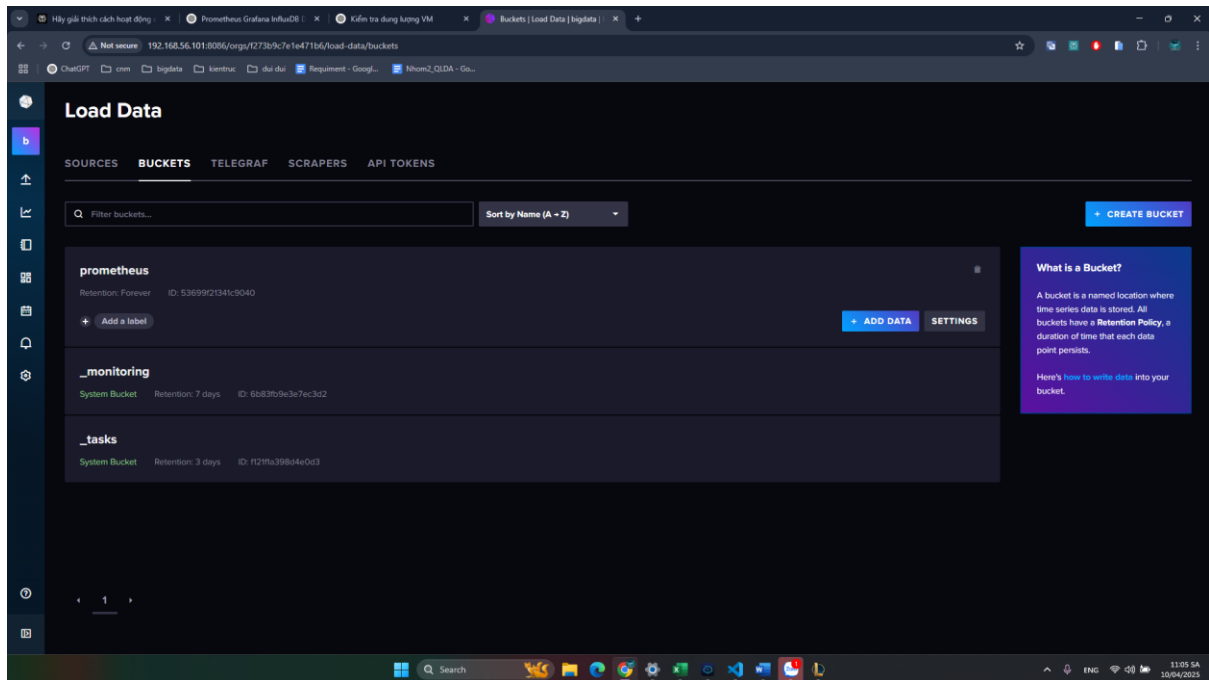
```
--mount type=volume,source=influxdb-data,target=/var/lib/influxdb \
```

```
--network prometheusnetwork \
```

```
influxdb:latest
```

```
sudo docker exec -it e1a0747a1b9a influx v1 shell
```

```
sudo docker config create prometheus_config ./prometheus-grafana/prometheus.yml  
p9w5tylojb389a839qd9xytzy
```



Cách tạo bucket và scaper metrics: cách 1 tạo trực tiếp trên UI (dễ r khỏi nói)

Cách 2 tạo bằng cli:

Tạo bucket

```
sudo docker exec -it e1a0747a1b9a influx bucket create -n cadvisor -o bigdata -t  
pfNuJ4HQ8bda52sLKK79-xNI_0aFEpQAUOVI-oWvtHzcaYSE4Rxy-  
0WGp8uZIW2KPJJ35XGIV196eYRO7M5OCw==
```

Tạo scaper:

```
sudo docker exec -it e1a0747a1b9a influx v1 dbrp create --db cadvisor --rp  
autogen -o bigdata --bucket-id 8cbb95a0fa5cdf6 --default -t  
pfNuJ4HQ8bda52sLKK79-xNI_0aFEpQAUOVI-oWvtHzcaYSE4Rxy-  
0WGp8uZIW2KPJJ35XGIV196eYRO7M5OCw==
```

Cho Scapper tiến hành kéo về bucket:

Cách lấy orgID: sudo docker exec -it e1a0747a1b9a influx org list -t pfNuJ4HQ8bda52sLKK79-xNI_0aFEpQAUOVI-oWvtHzcaYSE4Rxy-0WGp8uZIW2KPJJ35XGIV196eYRO7M5Ocw==

```
curl -X POST http://192.168.56.101:8086/api/v2/scrapers -H "Authorization: Token pfNuJ4HQ8bda52sLKK79-xNI_0aFEpQAUOVI-oWvtHzcaYSE4Rxy-0WGp8uZIW2KPJJ35XGIV196eYRO7M5Ocw==" -H "Content-Type: application/json" -d '{
```

```
  "name": "cadvisor",
  "bucketID": "8cbbe95a0fa5cdf6",
  "url": "http://192.168.56.105:8080/metrics",
  "orgID": "f273b9c7e1e471b6"
}'
```

```
ubuntu@ops4:~$ curl -X POST http://192.168.56.101:8086/api/v2/scrapers -H "Authorization: Token pfNuJ4HQ8bda52sLKK79-xNI_0aFEpQAUOVI-oWvtHzcaYSE4Rxy-0WGp8uZIW2KPJJ35XGIV196eYRO7M5Ocw==" -H "Content-Type: application/json" -d '{
  "name": "cadvisor",
  "bucketID": "8cbbe95a0fa5cdf6",
  "url": "http://192.168.56.105:8080/metrics",
  "orgID": "f273b9c7e1e471b6"
}'
{"id":"9eb3a2456c419900","name":"cadvisor","type":"","url":"http://192.168.56.105:8080/metrics","orgID":"f273b9c7e1e471b6","bucketID":"8cbbe95a0fa5cdf6","org":"bigdata","bucket":"cadvisor","links":{"self":"/api/v2/scrapers/9eb3a2456c419900","bucket":"/api/v2/buckets/8cbbe95a0fa5cdf6","organization":"/api/v2/orgs/f273b9c7e1e471b6","members":"/api/v2/scrapers/9eb3a2456c419900/members","owners":"/api/v2/scrapers/9eb3a2456c419900/owners"}}
```

Telegraf.conf:

[agent]

```
interval = "10s"
round_interval = true
metric_batch_size = 1000
metric_buffer_limit = 10000
collection_jitter = "0s"
flush_interval = "10s"
flush_jitter = "0s"
precision = ""
hostname = ""
omit_hostname = true
```

[[inputs.prometheus]]

```
urls = ["http://prometheus:9090/federate"]
```

```
name_prefix = "prometheus_"
params = { 'match[]' = ['{__name__=~".+"}'] }
```

```
[[outputs.influxdb]]
```

```
urls = ["http://influxdb:8086"]
```

```
database = "getAll"
```

```
username = "admin"
```

```
password = "password"
```

Tạo config:

```
docker config create telegraf_conf ./prometheus-grafana/telegraf.conf
```

Tạo telegraf:

```
sudo docker service create \
  --name telegraf \
  --config source=telegraf_conf,target=/etc/telegraf/telegraf.conf \
  --network prometheusnetwork \
  --mount type=bind,source=/etc/hostname,target=/etc/hostname,readonly \
  --mount type=bind,source=/etc/hosts,target=/etc/hosts,readonly \
  telegraf:1.27 \
  --config-directory /etc/telegraf
```

Lệnh để kiểm tra

```
sudo docker exec -it e1a0747a1b9a influx query 'from(bucket: "cadvisor") |>
range(start: -5m) |> limit(n:10)' -o bigdata -t pfNuJ4HQ8bda52sLKk79-
xNI_0aFEpQAUOVI-oWvtHzcaYSE4Rxy-
0WGp8uZIW2KPJJ35XGIV196eYRO7M5Ocw==
```

Truy cập shell:

```
sudo docker exec -it e1a0747a1b9a influx v1 shell -t pfNuJ4HQ8bda52sLKk79-
xNI_0aFEpQAUOVI-oWvtHzcaYSE4Rxy-
0WGp8uZIW2KPJJ35XGIV196eYRO7M5Ocw==
```

Chạy thử query

use "cadvisor"

>

> SELECT * from "cadvisor_version_info"

Interactive Table View (press q to exit mode, shift-up/down to navigate tables):

Name: cadvisor_version_info

Index	Time	cadvisorRevision	cadvisorVersion	gauge	kernelVersion	osVersion
1	1744269199173616896.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
2	174426920914324416.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
3	1744269219137992704.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
4	1744269229141108800.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
5	1744269239142182912.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
6	1744269249168897792.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
7	1744269259162056560.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
8	174426926913909456.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
9	1744269279142210560.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
10	1744269289146398528.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
11	1744269299145194768.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
12	1744269309143314688.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
13	1744269319136723456.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
14	1744269329133147904.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
15	1744269339151116800.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
16	1744269349143811456.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
17	1744269359138405376.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
18	174426936913878232.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
19	174426937912553696.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
20	174426938913809488.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
21	174426939913809456.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
22	1744269409196912384.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
23	1744269419147811584.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
24	174426942916791656.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
25	1744269439131715328.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
26	1744269449154565632.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
27	1744269459147348224.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
28	174426946913856928.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
29	1744269479147271936.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
30	1744269489136201872.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
31	1744269499156685312.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
32	174426950913399616.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
33	1744269519148619048.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
34	1744269529142123008.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
35	1744269539137715200.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
36	1744269549148224512.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
37	1744269559139230464.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18
38	1744269569143869952.000000000	6f3f25ba	v0.49.1	1.000000000	6.11.0-19-generic	Alpine Linux v3.18

7 Columns, 62 Rows, Page 1/2
Table 1/1, Statement 1/1

Nội dung thi còn sửa:

1. Giới thiệu các metric được thu thập trên toàn bộ các node

Trong cụm Docker Swarm, ta thường thu thập các metric sau thông qua **Node Exporter** và **cAdvisor**:

- **CPU**: Ví dụ node_cpu_seconds_total, container_cpu_usage_seconds_total
- **RAM**: Ví dụ node_memory_MemTotal_bytes, node_memory_MemAvailable_bytes, container_memory_usage_bytes
- **Disk**: Ví dụ node_filesystem_size_bytes, node_filesystem_avail_bytes, container_fs_usage_bytes
- **Process**: Số lượng process chạy, trạng thái process (node_procs_running)
- **File Descriptors & Sockets**: node_filefd_allocated, node_sockstat_TCP_tw
- **I/O**: Ví dụ node_disk_read_bytes_total, node_disk_written_bytes_total, node_network_receive_bytes_total, node_network_transmit_bytes_total

- **Phần cứng vật lý** (nếu có): Có thể thu thập thông tin như fan speed, cpu temperature (thường qua các exporter đặc thù hoặc cảm biến hệ thống như lm-sensors exporter)

```
curl -s http://192.168.56.104:9100/metrics | grep "^node_cpu_seconds_total"
```

2. Hiện thị thông tin metric trên cụm bằng lệnh CLI

Bạn có thể truy vấn Prometheus trực tiếp qua API hoặc xem dữ liệu trực tiếp từ các exporter.

Ví dụ dùng curl để query Prometheus:

- Hiện thị tất cả tên metric mà Prometheus đang scrape:

```
curl -s http://localhost:9090/api/v1/label/__name__/values | jq .
```

- Truy vấn metric RAM khả dụng từ Node Exporter:

```
curl -s "http://localhost:9090/api/v1/query?query=node_memory_MemAvailable_bytes" | jq .
```

- Truy vấn metric CPU container:

```
curl -s "http://localhost:9090/api/v1/query?query=container_cpu_usage_seconds_total" | jq .
```

Lưu ý: Thay localhost bằng IP hoặc hostname của instance Prometheus trong Swarm.

Ngoài ra, bạn có thể truy cập trực tiếp các exporter (để xem raw metrics):

- **Node Exporter (port 9100):**

```
curl http://192.168.56.104:9100/metrics | head -n 20
```

- **cAdvisor (port 8080):**

```
curl http://192.168.56.104:8080/metrics | head -n 20
```

3. Trình diễn cơ chế tạo pipeline thu thập và lưu trữ các metric với Prometheus qua CLI

Pipeline bao gồm:

1. **Scrape metrics**: Prometheus được cấu hình với prometheus.yml chứa các scrape_configs trỏ đến Node Exporter và cAdvisor.
2. **Lưu trữ cục bộ**: Prometheus sử dụng TSDB (thường lưu tại /prometheus trong container) để lưu trữ metrics.
3. **Remote Write** (nếu đồng bộ sang InfluxDB): Prometheus đẩy dữ liệu từ TSDB sang InfluxDB.

Cấu hình mẫu file prometheus.yml (InfluxDB v1.x hoặc v2 nếu dùng adapter – ở đây ta dùng v1 API cho ví dụ đơn giản):

yaml

global:

scrape_interval: 15s

scrape_configs:

- job_name: 'prometheus'

static_configs:

- targets: ['localhost:9090']

- job_name: 'node-exporter'

static_configs:

- targets: ['192.168.56.104:9100', '192.168.56.105:9100']

- job_name: 'cadvisor'

static_configs:

- targets: ['192.168.56.104:8080', '192.168.56.105:8080']

remote_write:

- url: "http://192.168.56.101:8086/api/v1/prom/write?db=prometheus"

basic_auth:

username: "admin"

password: "adminpassword"

Thực hành:

Sau khi cập nhật file, reload Prometheus để cấu hình mới có hiệu lực:

```
curl -X POST http://localhost:9090/-/reload
```

Hoặc dùng:

```
docker service update --force prometheus
```

4. Giới thiệu InfluxDB

InfluxDB là cơ sở dữ liệu dạng time-series chuyên dụng, được thiết kế để lưu trữ dữ liệu như metrics, logs, và sự kiện theo thời gian.

- **InfluxDB v1.x** dùng database truyền thống (ví dụ: prometheus) và hỗ trợ API `/write?db=....`
- Nó cho phép retention policy (chính sách lưu trữ dữ liệu) để xóa dữ liệu cũ sau một khoảng thời gian nhất định.
- InfluxDB có giao diện web, CLI và API REST để query dữ liệu.

5. Đồng bộ hóa dữ liệu từ Prometheus với InfluxDB

Để đồng bộ dữ liệu từ Prometheus sang InfluxDB v1, bạn cấu hình Prometheus remote_write như sau:

remote_write:

- url: "http://192.168.56.101:8086/api/v1/prom/write?db=prometheus"

basic_auth:

username: "admin"

password: "adminpassword"

- Với cấu hình này, Prometheus sẽ gửi dữ liệu được ghi nhận (WAL - Write Ahead Log) sang InfluxDB.
- **Thực hành:** Kiểm tra log của Prometheus để xem dòng "Done replaying WAL" để xác nhận dữ liệu đã được gửi.

6. Truy vấn dữ liệu metric được lưu trữ trong InfluxDB chạy trên Swarm

Nếu bạn đang sử dụng InfluxDB v1, bạn có thể dùng CLI hoặc curl để truy vấn.

Với CLI InfluxDB (v1):

1. Vào CLI:

```
influx -username admin -password adminpassword -database prometheus
```

2. Truy vấn dữ liệu:

```
sql
```

```
SHOW MEASUREMENTS;
```

```
SELECT * FROM "node_cpu_seconds_total" LIMIT 10;
```

Với curl:

```
curl -G "http://192.168.56.101:8086/query" \  
  --data-urlencode "db=prometheus" \  
  --data-urlencode "q=SELECT * FROM \"node_cpu_seconds_total\" LIMIT 10" \  
  -u admin:adminpassword
```

7. Giới thiệu công cụ Grafana

Grafana là công cụ visualization hàng đầu để tạo Dashboard dựa trên các nguồn dữ liệu như Prometheus và InfluxDB.

- Nó cho phép bạn tạo biểu đồ realtime, alerts và báo cáo.

- Hỗ trợ nhiều loại panel: time series, bar gauge, table, heatmap,...
 - Dễ dàng import/export dashboard dạng JSON.
-

8. Tạo Dashboard sử dụng Prometheus/Grafana để phân tích, giám sát hiệu suất Swarm

Các bước Demo:

1. Kết nối Data Source:

- Vào Grafana → Configuration → Data Sources
- Thêm Prometheus: URL = `http://<Prometheus_IP>:9090`
- Hoặc thêm InfluxDB (nếu muốn so sánh): URL = `http://192.168.56.101:8086`, Database = `prometheus`, User = `admin`, Password = `adminpassword`

2. Tạo Dashboard:

- Chọn "Create Dashboard"
- Thêm Panel hiển thị:

Các lệnh CLI tổng hợp Demo:

A. Kiểm tra Prometheus metrics qua API:

```
curl -s "http://localhost:9090/api/v1/query?query=up" | jq .
```

```
curl -s  
"http://localhost:9090/api/v1/query?query=avg(rate(node_cpu_seconds_total{mode!=  
'idle'}[5m])) by (instance)" | jq .
```

B. Reload cấu hình Prometheus:

```
curl -X POST http://localhost:9090/-/reload
```

hoặc

```
docker service update --force prometheus
```

C. Truy vấn InfluxDB bằng CLI (v1):

```
influx -username admin -password adminpassword -database prometheus -execute  
"SHOW MEASUREMENTS"
```

```
influx -username admin -password adminpassword -database prometheus -execute  
"SELECT * FROM \"node_cpu_seconds_total\" LIMIT 10"
```

D. Kiểm tra thông tin Data Source trong Grafana (API example):

```
curl -H "Authorization: Bearer <grafana_api_key>"  
http://<Grafana_IP>:3000/api/datasources
```

.